

Prompt Injection as Pipeline Architecture — one-pager

scripts/gen/osp_generate.py + osp_luna_generate.py · jac_llm_data · Sept 2026 · full report: docs/osp_prompt_injection_report.pdf

The OSP codegen pipeline **injects grounding material into the model’s context at runtime** from files, distilled skills, verified demos, and the model’s own compiler errors. Two injection sites:

Site 1 — import-time system-prompt surgery (osp_luna_generate.py:146-253; inherited by osp_generate.py via import, lines 88–91): six operations transform a 3,740-char “floor” prompt into the 14,077-char payload every call rides on — **73% is injected, not written**.

Layer	Chars
SYSTEM literal (floor prompt)	3,740
.replace(): ability ceiling → union triggers	+182
splice _SPEC[:4000] (OSP_IDIOMIZE_TASK.md)	+4,000
slice “## 6. The model task”	+1,705
_DEMO: verified demo (runs clean on jac 0.36.1)	+1,031
_FILTER_WARN: ban pre-0.36 filter syntax	+122
_SKILL_TIER0: distilled jac-idiomatic skill	+1,013
_COMPILER_PITFALLS: E0013/E0076/E0002/E0004	+602
REFERENCE RECORD: verified corpus file ≤3.5k	+1,622
injected total	10,337 (73%)

Site 2 — error-conditioned hint injection (runtime). Every gate failure re-prompts via _fix_user() (line 263): regex-scrape undefined name ‘X’ from the validator error and **inject a targeted repair hint** (add edge X { } or use plain ->); retry ≤ fix_tries=2. Failures + broken code log to osp_dataset_failures.jsonl as repair-training material.

Gates (same bar as the composer-2.5 reference batch):
extract_jac — single “jac fence

jac_check — jac check → jac run → stdout must be non-empty (silent pass = regression)

osp_contract — 8 regex gates: docstring, no Any, typed has, visit after typed entry, with entry, spawn, self-referential ternary, no __main__, undefined visit names

Measured: holdout probe — 10 problems failing the floor prompt 3× each went **0/10 → 3/10** with the injected context. Dataset: **7,275 records** (m3:free 4,970 · composer-2.5 1,308 · gpt-5.6-luna 975).

Containment: records persist only the raw problem + final code in messages — the injected context never enters the training transcript; provenance rides in meta-data (context_bundle_version etc.).

Risks: file splices mean editing OSP_IDIOMIZE_TASK.md silently changes generation (provenance = one meta-data field); layer order is precedence; the 3,500-char corpus cap truncates silently.

Inspect live: osp prompt -stat recomputes the table from the same module the pipeline uses; osp prompt dumps the full payload.

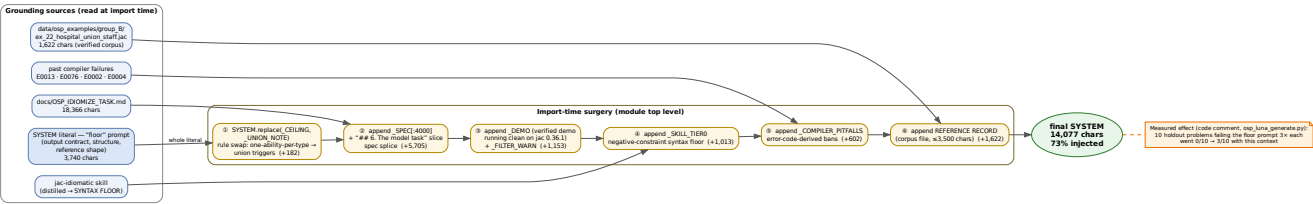


Fig. 1 — import-time injection stack.

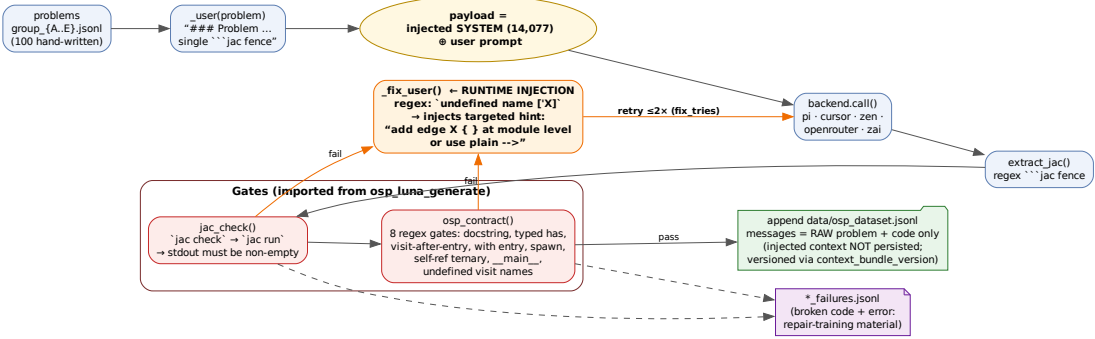


Fig. 2 — generate → gate → injected-hint repair loop.